

## Занятие 9

# Working with APIs in JavaScript

Как JavaScript общается с внешним миром через интернет — разбираем API, HTTP и инструменты для запросов.

ИНСТРУКТОР: АНДРЕЙ ПОМЕЛОВ







# Что такое API?

**API (Application Programming Interface)** — это способ взаимодействия между программами через интернет. Представьте официанта в ресторане: вы делаете заказ, он передаёт его на кухню и приносит результат. API работает точно так же — принимает запрос и возвращает нужные данные.

## Клиент

Ваш браузер или приложение, которое делает запрос

## API

Посредник, который принимает запрос и знает, как его обработать

## Сервер

База данных или сервис, который возвращает ответ



# HTTP — язык интернета

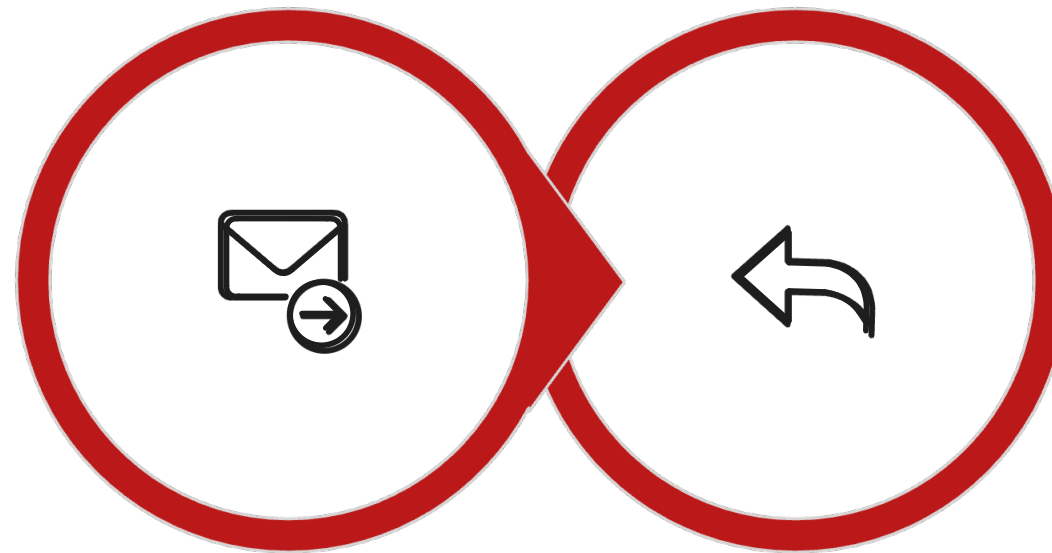


**HTTP (HyperText Transfer Protocol)** — протокол, по которому клиент и сервер обмениваются сообщениями. Каждый раз, когда вы открываете сайт или загружаете данные, браузер отправляет HTTP-запрос и ждёт ответа.

-  Думайте об HTTP как о языке, на котором говорят компьютеры в интернете. Без него программы не смогли бы «понять» друг друга.

# Request и Response

Любое общение с API строится по одной схеме: клиент отправляет **запрос (request)**, сервер возвращает **ответ (response)**.



**Запрос**

**Ответ**

Это похоже на SMS-переписку: вы пишете вопрос — получаете ответ. Сервер никогда не отвечает первым.

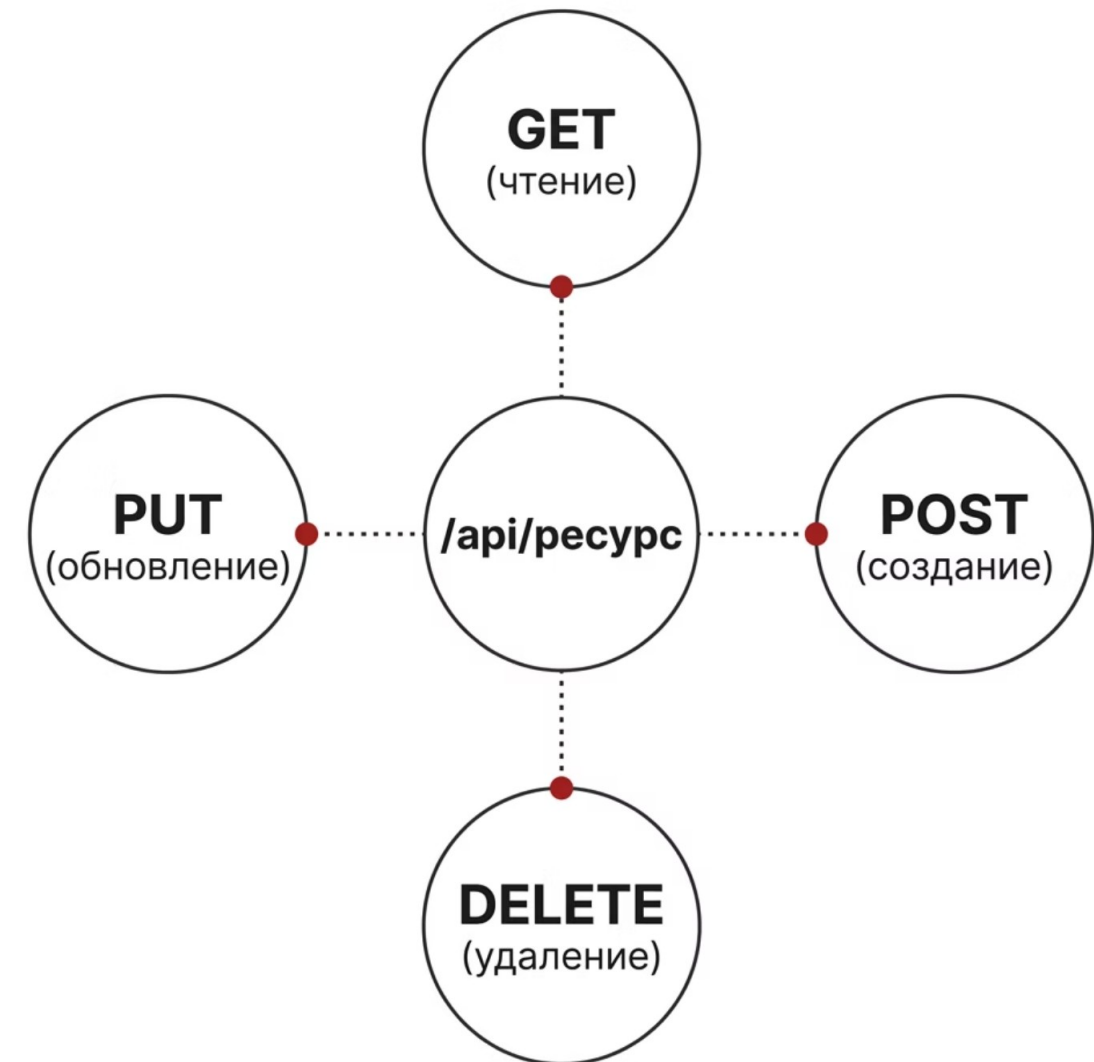




# REST — стандарт построения API

**REST (Representational State Transfer)** — это популярный подход к проектированию API. REST-API использует стандартные HTTP-методы, чтобы работать с ресурсами предсказуемо и понятно.

| Метод  | Действие        |
|--------|-----------------|
| GET    | Получить данные |
| POST   | Создать запись  |
| PUT    | Обновить запись |
| DELETE | Удалить запись  |



# fetch и Axios — инструменты для запросов

JavaScript предоставляет два популярных способа отправки HTTP-запросов:

## fetch

Встроен в браузер, не требует установки. Возвращает Promise. Минималистичный и стандартный.

## Axios

Сторонняя библиотека. Удобнее в работе с JSON, имеет лучшую обработку ошибок и более читаемый синтаксис.

📌 Для большинства учебных проектов **fetch** — отличный старт. Axios пригодится в реальных проектах с более сложной логикой.

# Простой GET-запрос на практике

Посмотрим, как выглядит запрос с помощью `fetch` к открытому API:

```
fetch('https://jsonplaceholder.typicode.com/posts/1')  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error('Ошибка:', error));
```

→ **fetch(url)** — отправляет GET-запрос по указанному адресу

→ **.then(response => response.json())** — преобразует ответ в JavaScript-объект

→ **.catch(error)** — перехватывает ошибки соединения или обработки



# Итог занятия

## API

Интерфейс для взаимодействия программ через интернет

## HTTP

Протокол передачи запросов и ответов между клиентом и сервером

## fetch / Axios

Удобные инструменты JavaScript для работы с внешними API

✅ Теперь вы знаете, как JavaScript получает данные из внешнего мира.

